Public - عام

Lab 04

Course: CSC 402 — Mobile Application Programming
Student name: Elaf hassan al nutayf
Student ID: 2240005931

This report presents the four guided tasks for Lab04Attendance and the individual StudyPlanner Semester App Part 1. Both projects use immutable UI state, stateless screen composables, previews, and a Route that owns and updates the state.

Task 1 Find the way in

1.1 Activity name

The value of android in the activity element is `.MainActivity`.

1.2 Launcher declarations

The two declarations inside the intent filter are:

```
<action android:name="android.intent.action.MAIN" />

<category android:name="android.intent.category.LAUNCHER" />
```

MAIN identifies the Activity as an entry point. LAUNCHER makes it available from the app launcher.

1.3 Operations inside onCreate

The following calls occur in order:

1. `super.onCreate(savedInstanceState)` calls the superclass implementation.
2. `enableEdgeToEdge()` enables edge-to-edge display.
3. `setContent { ... }` defines the Compose interface. Inside the application theme, it calls `AttendanceRoute()`.

1.4 Removing the launcher category

Removing the LAUNCHER category makes the app's Activity unavailable from the app drawer. The package can still be installed because installation requires a valid package, not a launcher entry.

1.5 Restoring the launcher category

Restoring the LAUNCHER category makes the Activity available from the app launcher again. The final manifest includes both MAIN and LAUNCHER.

Public - عام

Task 2 Name the state

2.1 What the screen shows

The screen displays:

- The session title.
- The number of students who have checked in.
- A Check in button that becomes disabled after the current student checks in.

2.2 What can change

The checked-in count increases when the student checks in. The student's check-in status changes from false to true, which disables the button.

The session title is also included in the state model so the screen receives it as data. It remains unchanged during this interaction.

2.3 User action

The user can tap Check in. The event callback is named `onCheckIn`.

2.4 State class

AttendanceUiState contains these properties:

- `val sessionTitle: String = ""`
- `val checkedInCount: Int = 0`
- `val isCheckedIn: Boolean = false`

It is a data class with immutable properties. Every property has a default value, so `AttendanceUiState()` creates a valid initial state.

Task 3 Write and preview the screen

3.1 Completing the stateless screen

AttendanceScreen receives three parameters:

- `uiState: AttendanceUiState`
- `onCheckIn: () -> Unit`
- `modifier: Modifier = Modifier`

The modifier is the last parameter and has a default value.

The count is displayed using:

```
Text("Checked in: ${uiState.checkedInCount}")
```

The Check in button uses `onClick = onCheckIn` to report the tap to its caller. Its enabled value is `!uiState.isCheckedIn`.

AttendanceScreen does not contain remember or mutableStateOf. It displays the supplied state and sends the user's action through the callback.

3.2 to 3.4 Preview states

The Not checked in yet preview uses:

```
AttendanceUiState("CSC 402 - Week 4", 12, false)
```

The Already checked in preview uses:

```
AttendanceUiState("CSC 402 - Week 4", 12, true)
```

Both previews use Lab04AttendanceTheme and an empty onCheckIn callback.

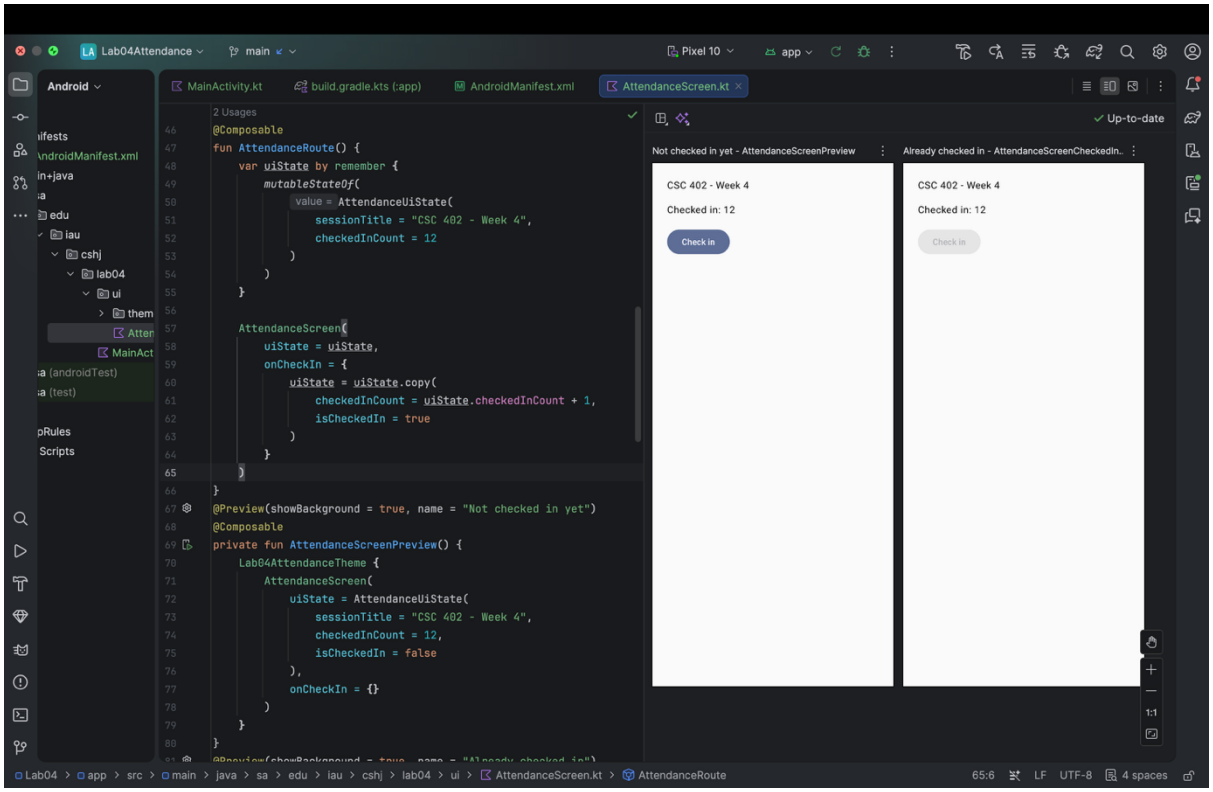


Figure 1. AttendanceScreen previews showing the enabled and disabled Check in button.

3.5 Difference between the previews

The button is enabled in the first preview and disabled in the second. The only changed field is isCheckedIn. The session title and checked-in count remain the same.

The screen can be previewed with different states because its data is supplied through parameters.

Task 4 Make it work and break it

4.1 State ownership

AttendanceRoute owns uiState using remember and mutableStateOf. Its initial state contains:

- Session title: CSC 402 - Week 4
- Checked-in count: 12
- Check-in status: false

When onCheckIn is invoked, the Route creates a new state using copy and assigns it to uiState. The new state increases checkedInCount by one and sets isCheckedIn to true.

4.2 Connecting the Route

MainActivity calls AttendanceRoute inside setContent and Lab04AttendanceTheme.

4.3 Running the application

After one tap on Check in, the displayed count changes from 12 to 13 and the button becomes disabled.

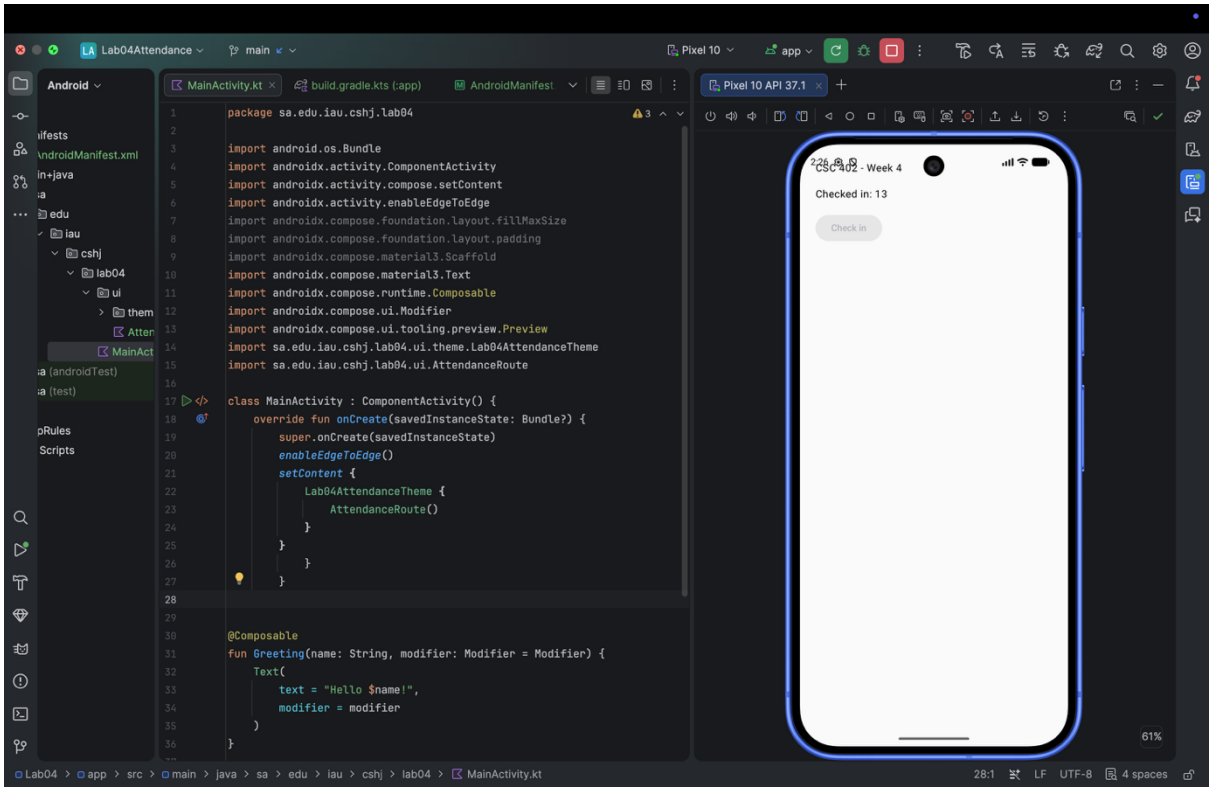


Figure 2. The running attendance app after one check-in.

4.4 Trace of one tap

Public - عام

Step	What happens	Function or component
1	The user taps Check in.	AttendanceScreen
2	The button invokes the supplied onCheckIn callback.	AttendanceScreen
3	The callback creates and assigns a new state with count 13 and isCheckedIn set to true.	AttendanceRoute
4	Compose observes the state change and recomposes the affected UI.	Compose runtime
5	The screen displays the updated count and disables the button.	AttendanceScreen

4.5 Breaking the event handler

For the temporary broken version, the event handler contains only:

```
uiState.checkedInCount + 1
```

4.6 Result and explanation

After a tap, the count remains 12 and the button remains enabled.

The expression calculates a value but discards it. It does not assign a new state to uiState. Step 3 of the trace therefore fails, and Compose has no updated state to display.

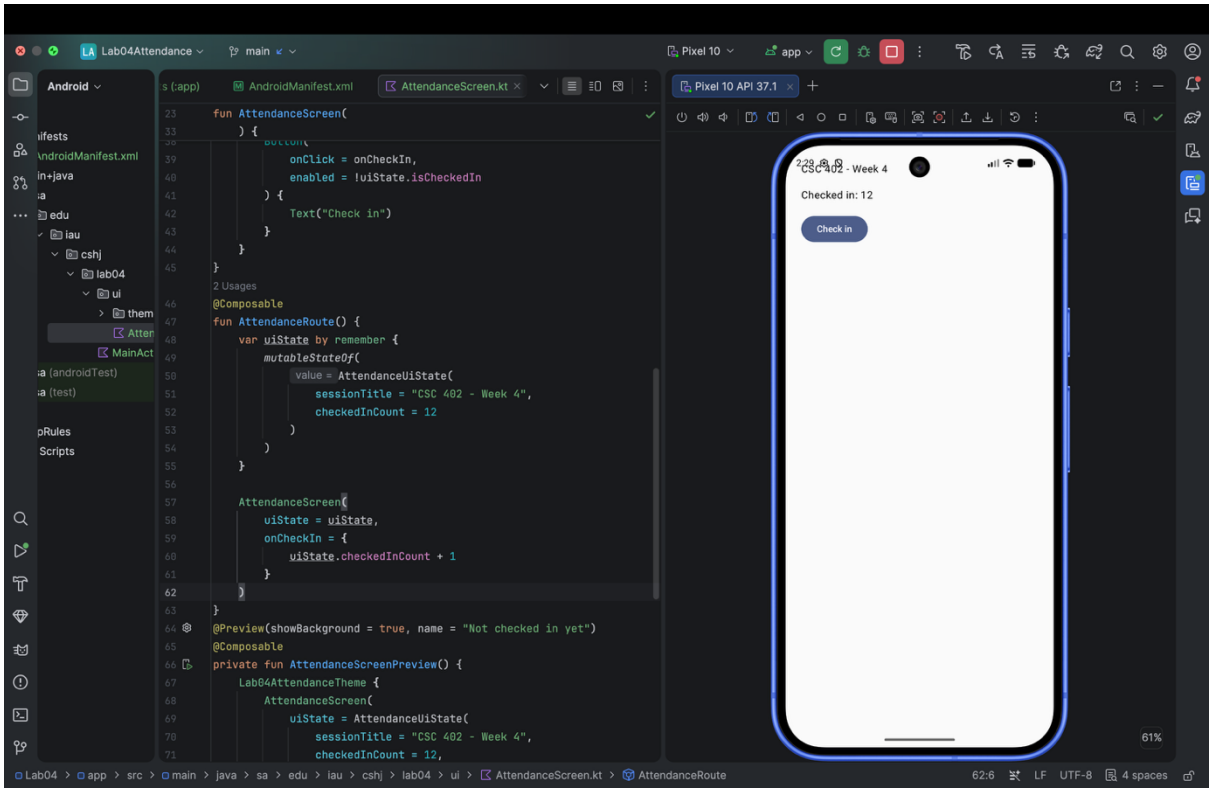


Figure 3. The temporary broken handler and the unchanged attendance screen.

4.7 Restoring the working implementation

Public - عام

The working handler assigns the result of copy to uiState, increases checkedInCount, and sets isCheckedIn to true. This is the version used for the final submission.

Semester App Part 1 StudyPlanner

P1.1 Paper sketch and state list

The home screen shows:

- A greeting.
- A progress line showing completed tasks out of the total.
- A Mark one done button.

The state fields are greeting, totalTasks, and doneTasks. During the Part 1 button interaction, doneTasks changes while greeting and totalTasks remain unchanged.

The user action is named `onMarkDone`.

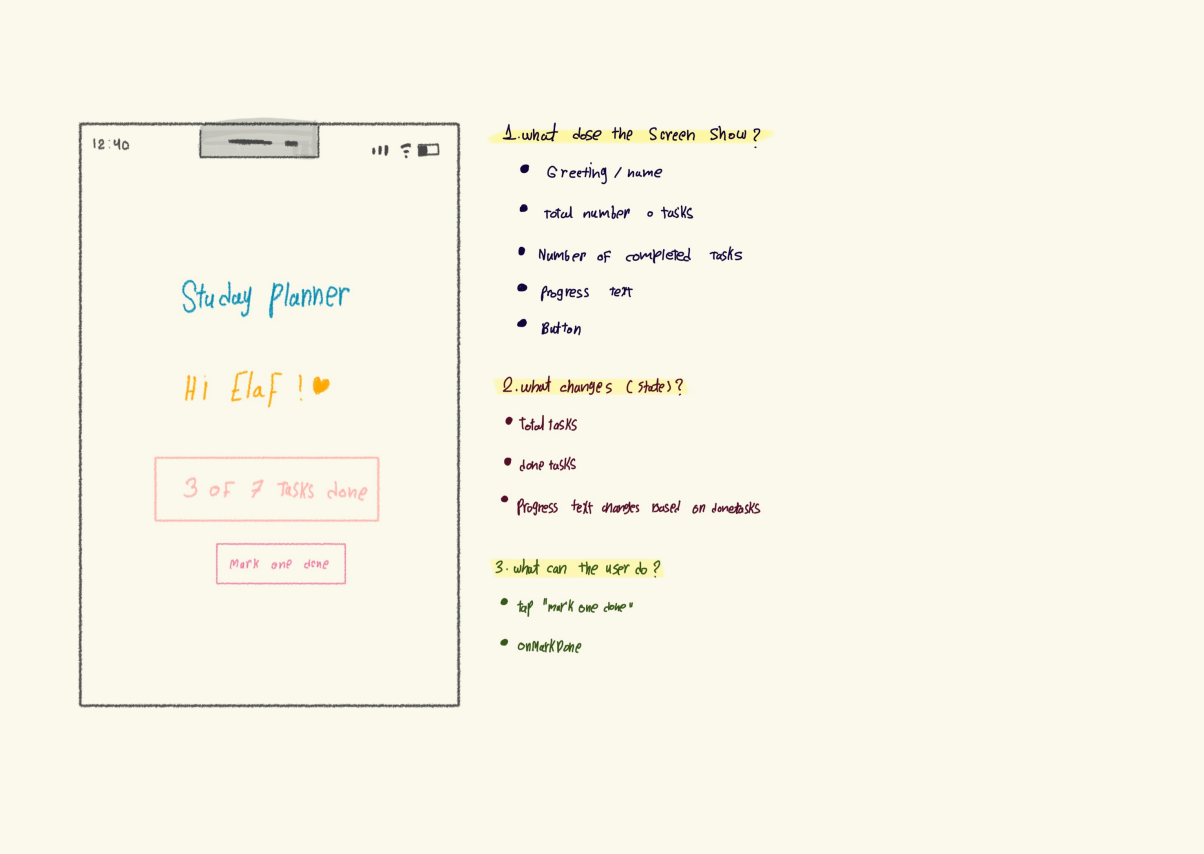


Figure 4. The StudyPlanner paper sketch and state list.

P1.2 Project configuration

Setting	Value
Project name	StudyPlanner
Package name	sa.edu.iau.cshj.studyplanner

Language	Kotlin
User interface	Jetpack Compose
Minimum SDK	API 26

P1.3 Immutable UI state

PlannerUiState is a data class containing:

- `val greeting: String = "Hi Elaf!"`
- `val totalTasks: Int = 7`
- `val doneTasks: Int = 3`

All properties use val and have default values.

P1.4 Stateless home screen

PlannerHomeScreen receives PlannerUiState and an onMarkDone event callback. Its last parameter is `modifier: Modifier = Modifier`.

The screen displays the greeting, task progress, and a Mark one done button. It holds no mutable state of its own. The button invokes onMarkDone, allowing the caller to handle the state update.

The UI uses Material 3 components.

P1.5 Two different previews

The Nothing done yet preview displays 0 of 7 tasks done.

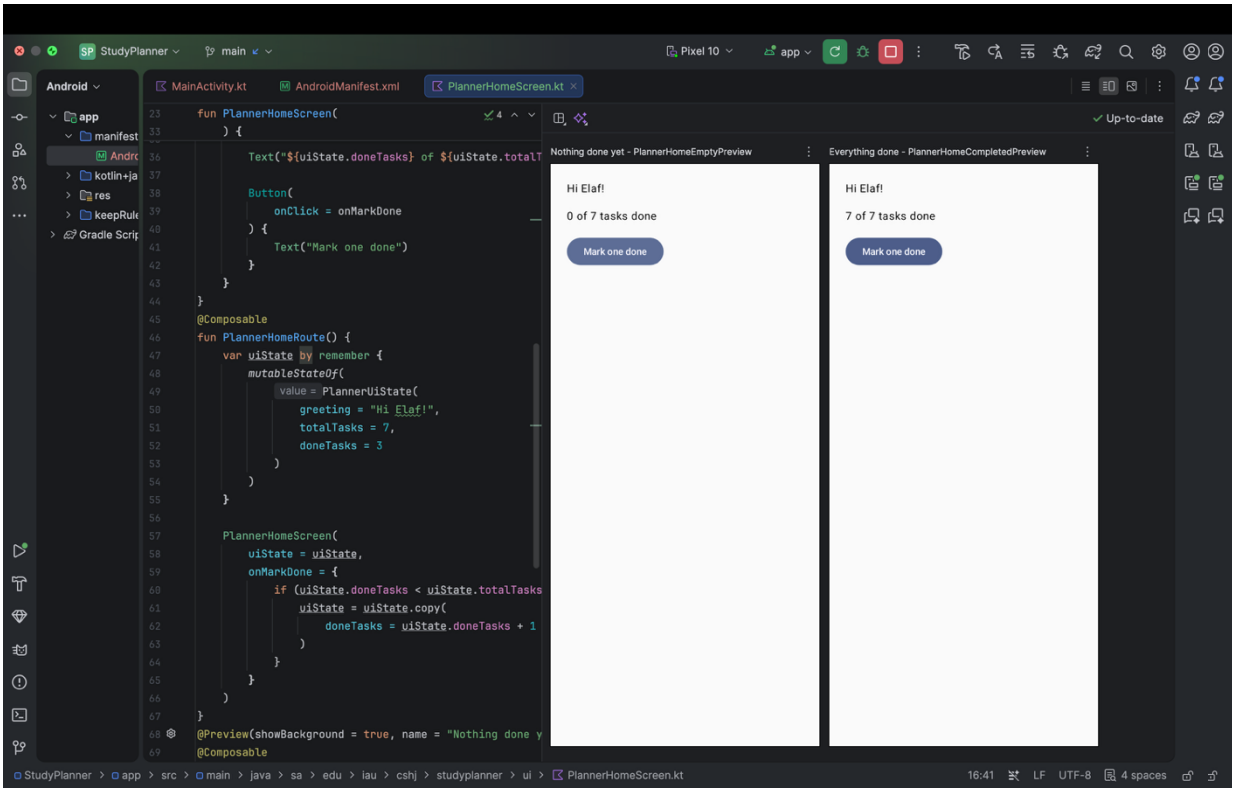


Figure 5. StudyPlanner with no completed tasks.

The Everything done preview displays 7 of 7 tasks done.

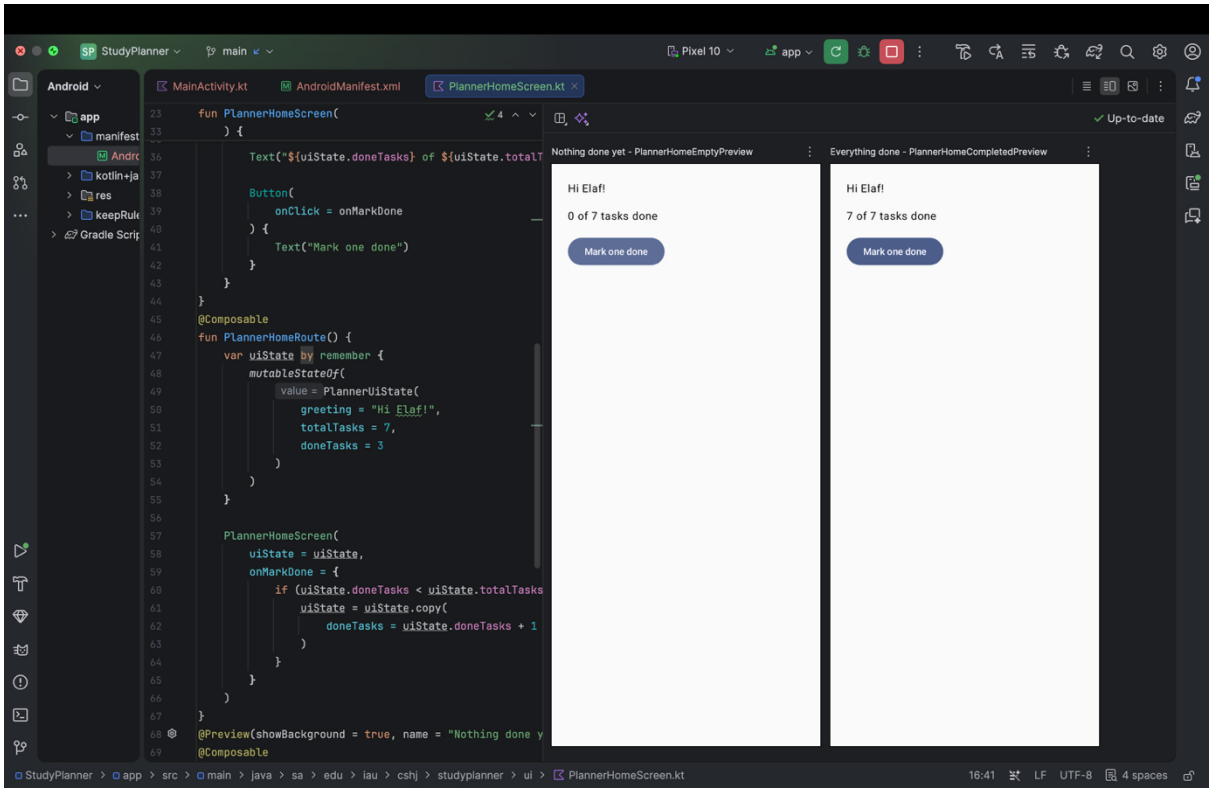


Figure 6. StudyPlanner with all tasks completed.

These previews demonstrate two different states supplied to the same stateless screen.

P1.6 Route and button interaction

PlannerHomeRoute owns the state using remember and mutableStateOf. It starts with 3 completed tasks out of 7.

When the button is tapped, the Route checks whether doneTasks is less than totalTasks. If it is, the Route creates a new state using copy, increases doneTasks by one, and assigns the result to uiState.

The condition prevents the completed count from exceeding the total number of tasks.

MainActivity calls PlannerHomeRoute inside setContent and StudyPlannerTheme. After one tap, the running app displays 4 of 7 tasks done.

Public - عام

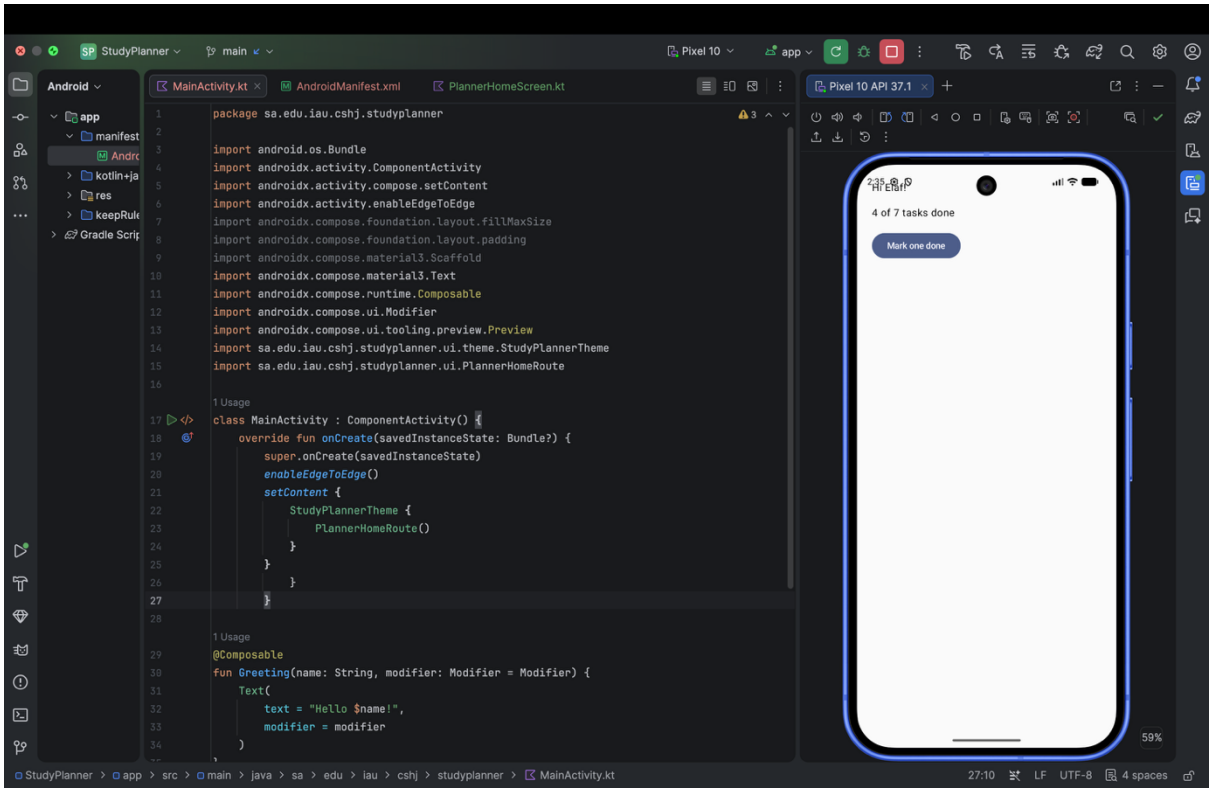


Figure 7. StudyPlanner after tapping Mark one done once.

P1.7 Manifest configuration

The manifest declares exactly one Activity, `.MainActivity`. Its intent filter includes MAIN and LAUNCHER, and the app label is StudyPlanner.

Public - عام